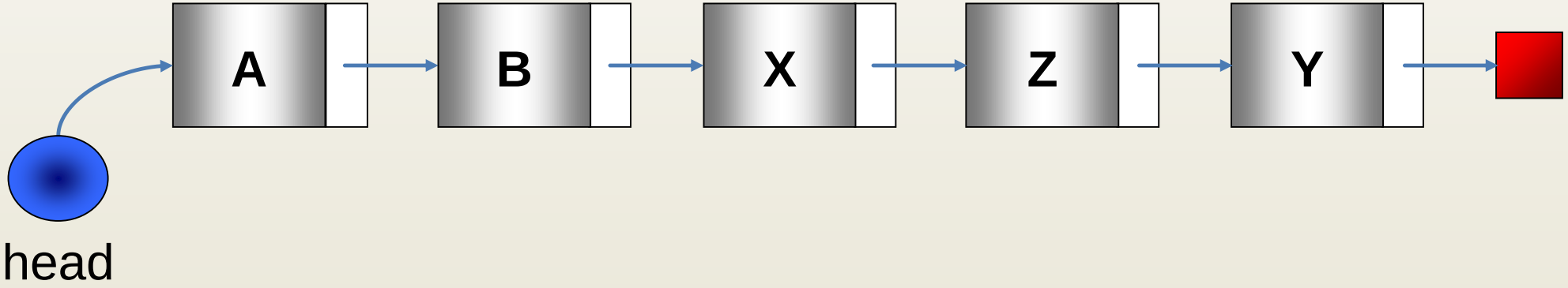
# Danh sách liên kết

# Nội dung

1. Danh sách liên kết đơn
    1. Khai báo
    2. Các thao tác cơ sở
2. Danh sách liên kết vòng
3. Danh sách liên kết kép
    1. Khai báo
    2. Các thao tác cơ sở
4. Cấu trúc đa liên kết

# Danh sách liên kết đơn



# Khai báo trong C/C++

```
1. struct TNode
2. {
3.     DataStruct data;   //dữ liệu của node
4.     struct TNode *pnext;   //địa chỉ phần tử kế
5. };
6. struct TSList
7. {
8.     TNode *phead; //địa chỉ phần tử đầu danh sách
9.     TNode *ptail; //địa chỉ phần tử cuối danh sách, nếu cần
10.};
```

# Các thao tác cơ sở

1. Hàm bổ trợ: tạo vùng nhớ lưu trữ 1 node
2. Khởi tạo danh sách
3. Thêm một phần tử vào đầu/cuối hoặc chèn vào danh sách
4. Duyệt danh sách
5. Tìm kiếm trên danh sách
6. Xóa một phần tử khỏi danh sách
7. Xóa toàn bộ danh sách
8. Nối 2 danh sách

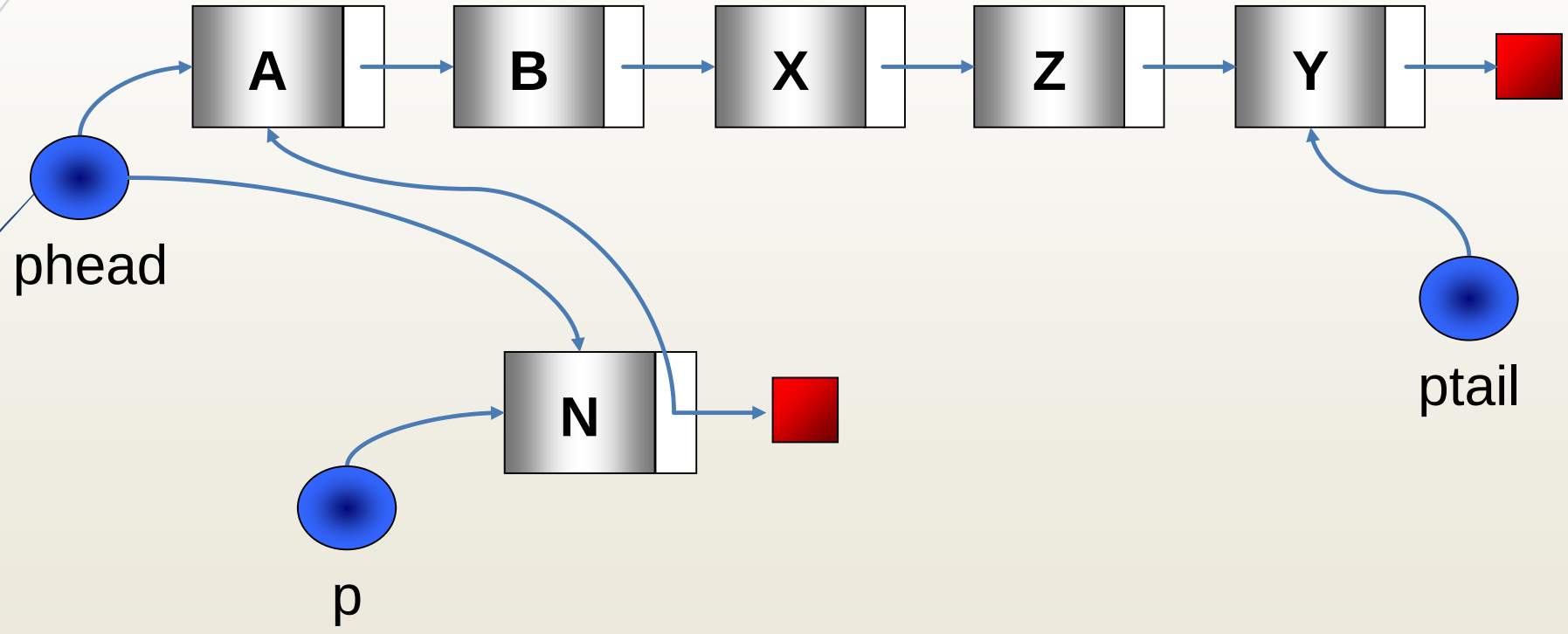
# Cấp vùng nhớ cho 1 phần tử

```
TNode *NewNode(DataStruct d)
{
    TNode *p = new TNode;
    if (!p) return NULL;
    p->data = d;
    p->pnext = NULL;
    return p;
}
```

# Khởi tạo, kiểm tra danh sách rỗng

```
1. void InitSList(TSList &slist)
2. {
3.     slist.phead = slist.ptail = NULL;
4. }

5. bool IsEmpty(TSList slist)
6. {
7.     return (!slist.phead);
8. }
```

# Thêm node vào đầu danh sách



# Thêm node vào đầu danh sách

```
1. void AddFirst(TSList &slist, TNode *p)
2. {
3.     if (IsEmpty(slist))    slist.phead = slist.ptail = p;
4.     else
5.     {
6.         p->pnext = slist.phead;
7.         slist.phead = p;
8.     }
9. }
```

# Thêm phần tử vào đầu danh sách

```
bool InsertFirst(TSList &slist, DataStruct d)
{
    TNode *p = NewNode(d);
    if (!p)  return false;
    AddFirst(slist, p);
    return true;
}
```

# Thêm node vào cuối danh sách

```
1. void AddTail(TSList &slist, TNode *p)
2. {
3.     if (IsEmpty(slist))
4.         slist.phead = slist.ptail = p;
5.     else
6.         slist.ptail = slist.ptail->pnext = p;
7. }
```

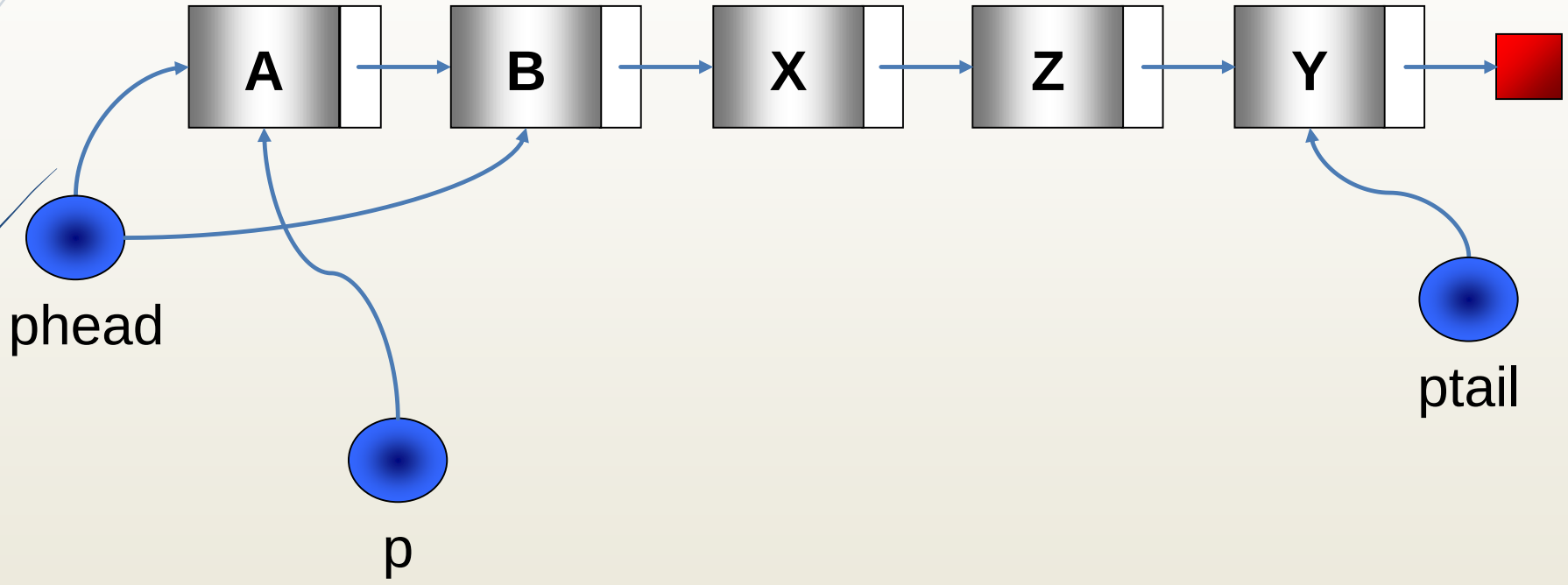
# Chèn node vào sau node q

```
void InsertAfter(TSList &slist, TNode *q, TNode *p)
{
    if (!q)  AddFirst(slist, p);
    else
    {
        p->pnext = q->pnext;
        q->pnext = p;
        if (q == slist.ptail) //update tail
            slist.ptail = p;
    }
}
```

# Tìm phần tử x trong danh sách

```
TNode* Search(TSList &slist, DataStruct x)
{
    TNode *p;
    for (p = slist.phead; p && (p->data != x);
                                        p=p->pnext);
    return p;
}
```

# Hủy node vào đầu danh sách



# Xóa phần tử ở đầu danh sách

```
bool RemoveHead(TSList &slist)
{
    if (IsEmpty(slist))
        return false;
    TNode *p = slist.phead;
    slist.phead = p->pnext;
    if (!slist.phead)
        slist.ptail = NULL;
    delete p;
    return true;
}
```

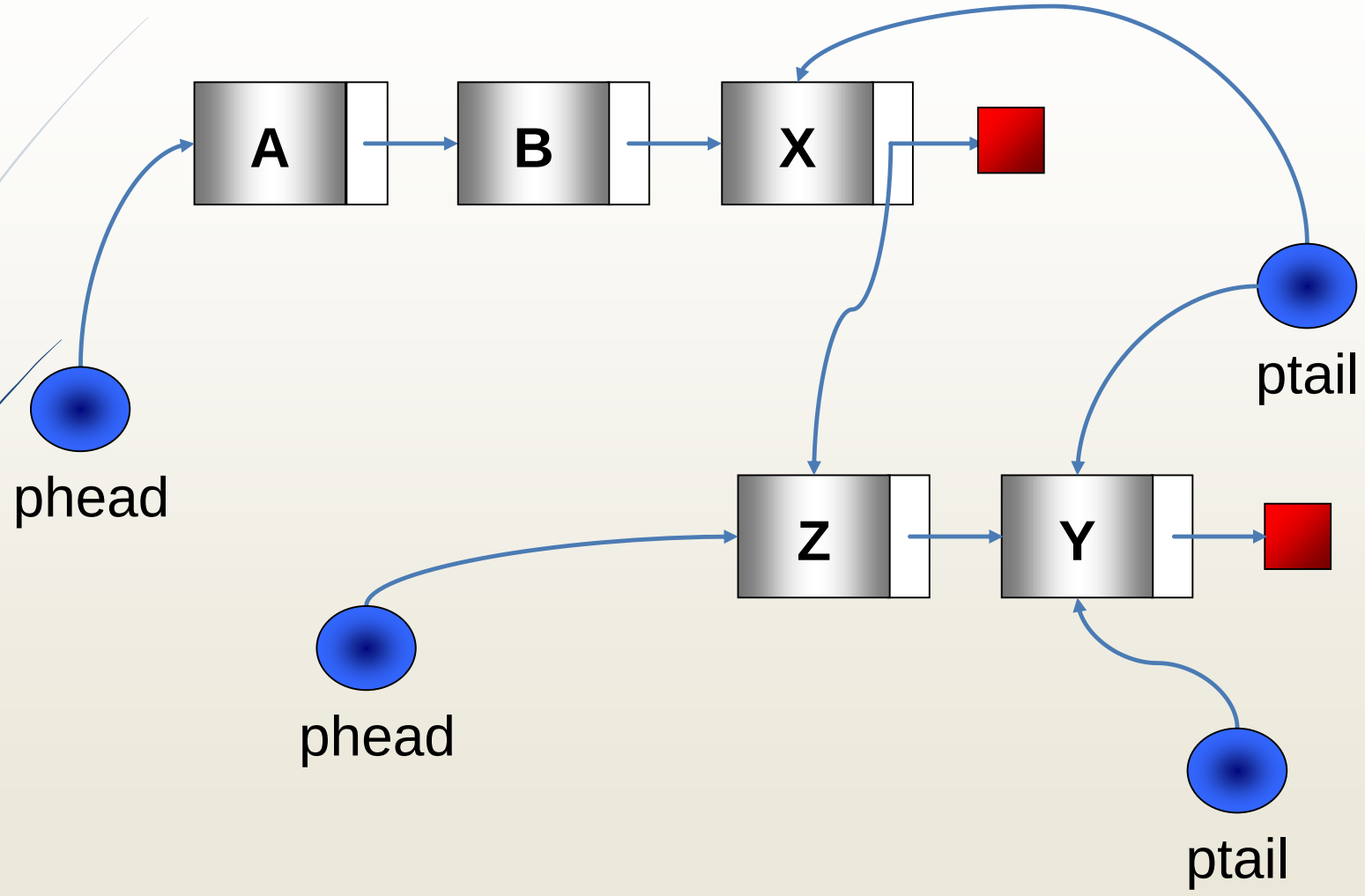
# Hàm loại bỏ x khỏi danh sách

```
1. bool RemoveX(TSList &slist, DataStruct x)
2. {
3.     if (!slist.phead) return false;
4.     if (slist.phead-> data == x)
5.         return RemoveHead(slist);
6.     TNode *p, *prev = slist.phead;
11.    for (p = prev->pnext; p && p->data != x;
12.                                prev = p, p=p->pnext);
13.    if (!p)     return false;
14.    prev->pnext = p->pnext;
15.    p->pnext = NULL;
16.    delete p;
17.    return true;
18.}
```

# Xóa toàn bộ danh sách

```
1. void RemoveAll(TSList &slist)
2. {
3.     TNode *p;
4.     while (slist.phead)
11.    {
12.        p = slist.phead;
13.        slist.phead = p->pnext;
1.         p->pnext = NULL;
2.         delete p;
3.     }
4.     slist.ptail = NULL;
5. }
```

# Nối danh sách $s_2$ vào sau $s_1$



# Nối danh sách $s_2$ vào sau $s_1$

```
void Concatenate(TSList &s1, TSList &s2)
{
    if (IsEmpty(s2)) return;
    if (IsEmpty(s1))
        s1 = s2;
    else
    {
        s1.ptail->pnext = s2.phead;
        s1.ptail = s2.ptail;
    }
}
```

# Sắp xếp danh sách liên kết đơn

Các thuật toán làm việc hiệu quả với danh sách đơn

1. Quick Sort
2. Merge Sort
3. Radix Sort

# Ví dụ: biểu diễn đa thức

```
struct TMonomial
{
    float factor;
    int degree;
};
struct TNode
{
    TMonomial monomial; //dữ liệu của node – đơn thức
    struct TNode *pnext;   //địa chỉ phần tử kế
};
```

# Ví dụ: biểu diễn đa thức

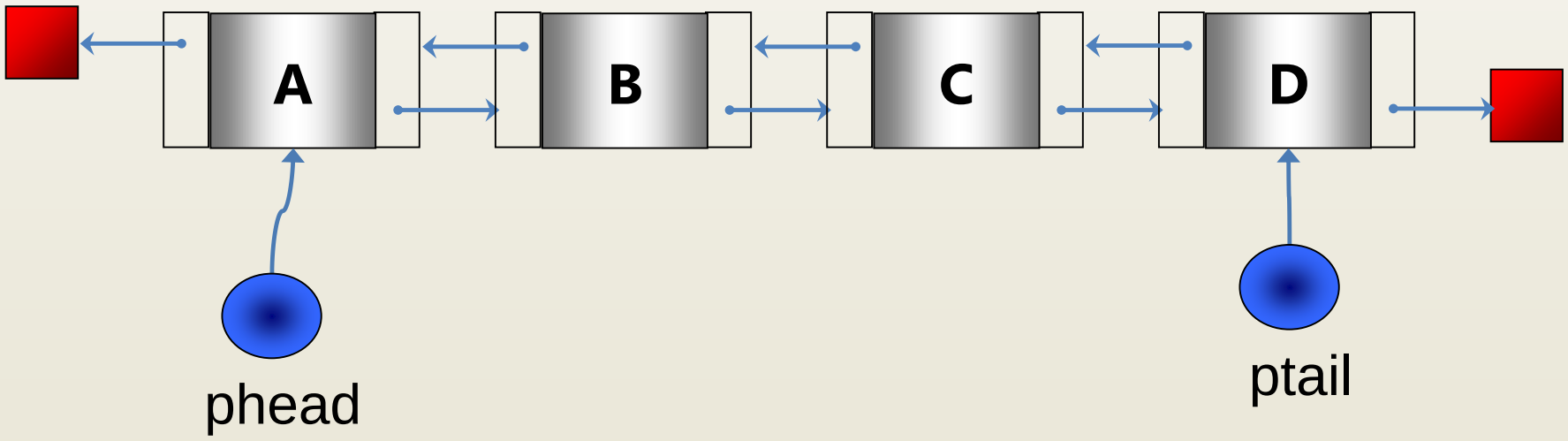
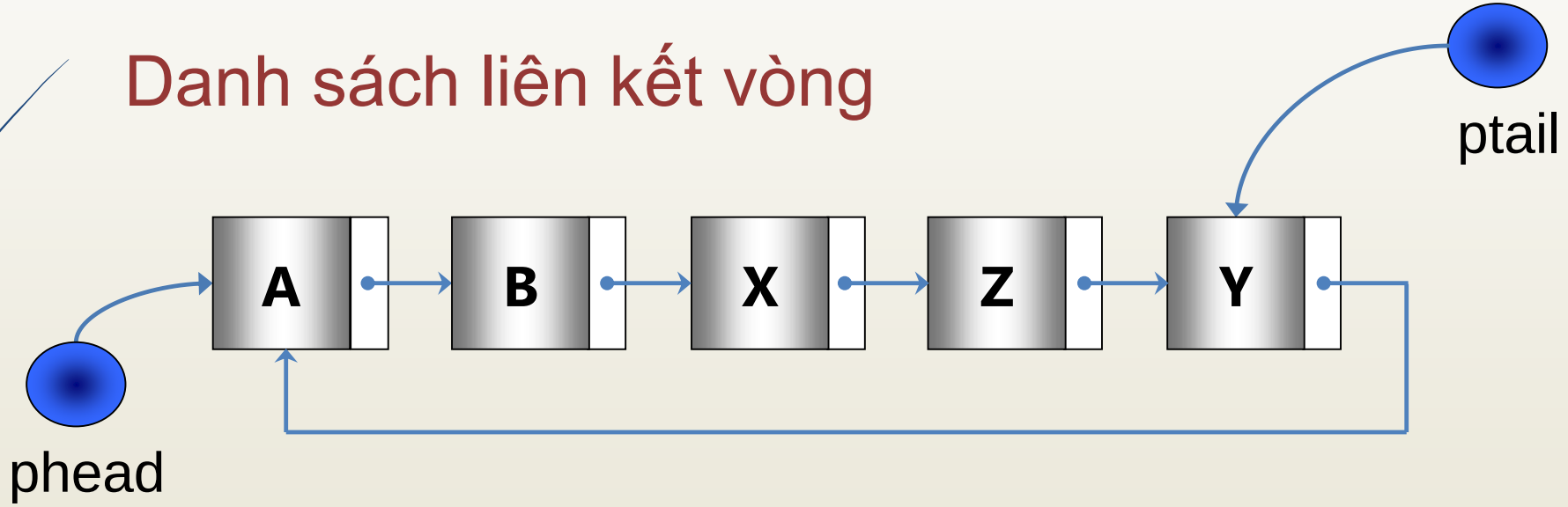
```
struct TPolynomial
{
    TNode *phead; //địa chỉ đơn thức đầu tiên trong đa thức
    TNode *ptail; //địa chỉ đơn thức cuối cùng trong đa thức
};

TPolynomial  polynomial;  //biến quản lý đa thức
```

# Bài tập về nhà

Xây dựng chương trình làm việc với đa thức được biểu diễn bằng danh sách liên kết đơn, chương trình có các chức năng:

1. Nhập mới 1 đa thức
2. Xác định bậc của đa thức
3. Tính giá trị của đa thức tại điểm x, x được nhập từ bàn phím
4. Tính tổng của đa thức đang có với đa thức p, p được nhập từ bàn phím

# Danh sách liên kết kép



# Danh sách liên kết vòng